EXP 4: Geometric Transformations Using OpenCV

Name: SAAADHANA A

Reg no: 212225240126

In [16]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

In [17]:
```
# Step 1: Load the image
image = cv2.imread('pic .jpg')  # Load the image from file
```

In [18]:
```
# Display the original image
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))  # Convert BGR to RGB for correc
plt.title("Original Image")
plt.axis('off')
```

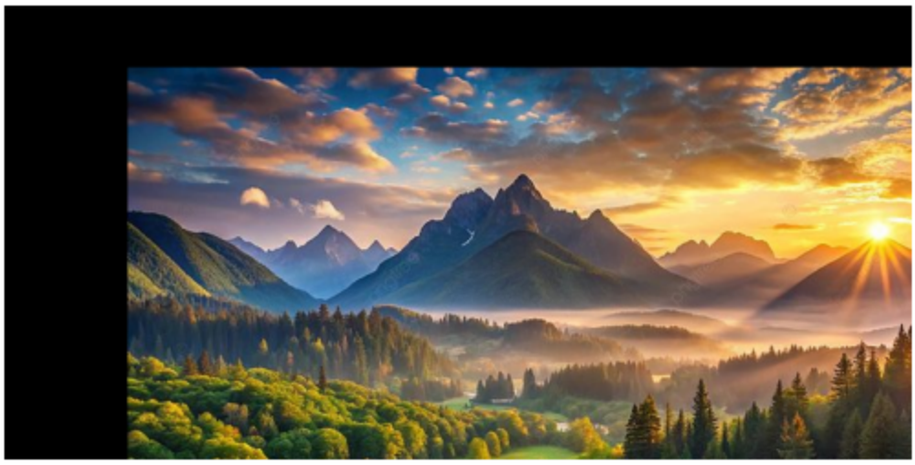
Out[18]: (np.float64(-0.5), np.float64(735.5), np.float64(367.5), np.float64(-0.5))

In [19]:
```
# Step 2: Image Translation
tx, ty = 100, 50  # Translation factors (shift by 100 pixels horizontally and 50 ve
M_translation = np.float32([[1, 0, tx], [0, 1, ty]])  # Translation matrix:
# [1, 0, tx] - Horizontal shift by tx
# [0, 1, ty] - Vertical shift by ty
translated_image = cv2.warpAffine(image, M_translation, (image.shape[1], image.shap
```

In [20]:
```
plt.imshow(cv2.cvtColor(translated_image, cv2.COLOR_BGR2RGB))  # Display the transl
plt.title("Translated Image")
plt.axis('off')
```

Out[20]: (np.float64(-0.5), np.float64(735.5), np.float64(367.5), np.float64(-0.5))

Translated Image



In [21]:
```
# Step 3: Image Scaling
fx, fy = 5.0, 2.0  # Scaling factors (1.5x scaling for both width and height)
scaled_image = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR
# resize: Resize the image by scaling factors fx, fy
# INTER_LINEAR: Uses bilinear interpolation for resizing
```

In [22]:
```
plt.imshow(cv2.cvtColor(scaled_image, cv2.COLOR_BGR2RGB))  # Display the scaled ima
plt.title("Scaled Image")  # Set title
plt.axis('off')
```

Out[22]:
```
(np.float64(-0.5), np.float64(3679.5), np.float64(735.5), np.float64(-0.5))
```

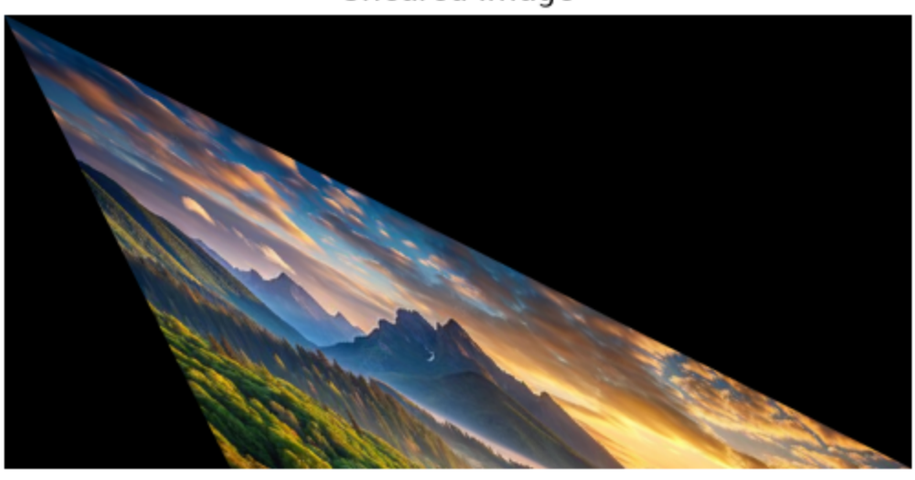
Scaled Image

In [23]:
```
# Step 4: Image Shearing
shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]])  # Shearing matrix
# The matrix shears the image by a factor of 0.5 in both x and y directions
# [1, 0.5, 0] - Shear along the x-axis (horizontal)
# [0.5, 1, 0] - Shear along the y-axis (vertical)
sheared_image = cv2.warpAffine(image, shear_matrix, (image.shape[1], image.shape[0]
```

In [24]:
```
plt.imshow(cv2.cvtColor(sheared_image, cv2.COLOR_BGR2RGB))  # Display the sheared i
plt.title("Sheared Image")  # Set title
plt.axis('off')
```

Out[24]:
```
(np.float64(-0.5), np.float64(735.5), np.float64(367.5), np.float64(-0.5))
```

Sheared Image



In [25]:
```
# Step 5: Image Reflection
reflected_image = cv2.flip(image, 2)  # Flip the image horizontally (1 means horizo
# flip: 1 means horizontal flip, 0 would be vertical flip, -1 would flip both axes
```

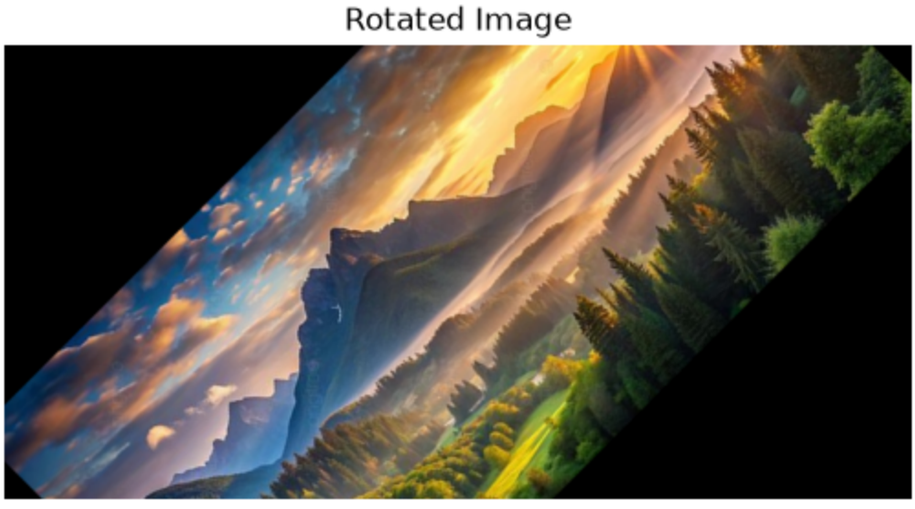
In [26]:
```
plt.imshow(cv2.cvtColor(reflected_image, cv2.COLOR_BGR2RGB))  # Display the reflect
plt.title("Reflected Image")  # Set title
plt.axis('off')
```

Out[26]: (np.float64(-0.5), np.float64(735.5), np.float64(367.5), np.float64(-0.5))

In [27]:
```
# Step 6: Image Rotation
(height, width) = image.shape[:2]  # Get the image height and width
angle = 45  # Rotation angle in degrees (rotate by 45 degrees)
center = (width // 2, height // 2)  # Set the center of rotation to the image cente
M_rotation = cv2.getRotationMatrix2D(center, angle, 1)  # Get the rotation matrix
# getRotationMatrix2D: Takes the center of rotation, angle, and scale factor (1 mea
rotated_image = cv2.warpAffine(image, M_rotation, (width, height))  # Apply rotatio
```

In [28]:
```
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))  # Display the rotated i
plt.title("Rotated Image")  # Set title
plt.axis('off')
```

Out[28]: (np.float64(-0.5), np.float64(735.5), np.float64(367.5), np.float64(-0.5))



In [29]:
```
# Step 7: Image Cropping
x, y, w, h = 100, 100, 200, 150  # Define the top-left corner (x, y) and the width
# Cropping the image from coordinates (x, y) to (x+w, y+h)
cropped_image = image[y:y+h, x:x+w]
# The crop is performed by slicing the image array in the y and x directions
```

In [32]:
```
plt.imshow(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB))  # Display the cropped i
plt.title("Cropped Image")  # Set title
plt.axis('off')
```

Out[32]: (np.float64(-0.5), np.float64(199.5), np.float64(149.5), np.float64(-0.5))

In [ ]: